

1. labor gyakorlat: rendszer programozáshoz használt eszközök

Tartalom

1. A GNU C fordító: gcc.....	1
1.1. A gcc fordítóprogram használata.....	1
1.1.1. C könyvtárak használata.....	3
1.1.2. Fejléc állományok használata.....	4
1.1.3. Néhány fontos gcc opció.....	4
2. Programok fordításának automatizálása: a make parancs.....	6
3. A C program parancssori paramétereinek használata.....	6
4. Unix szövegszerkesztők C programozáshoz.....	6
4.1. A vi és vim használata C programok szerkesztésére.....	7
4.2. Az Emacs használata C programok fejlesztéséhez.....	7
4.3. Fejlesztői környezetek használata C programozáshoz.....	8
4.4. A Code::Blocks beállítása C programokhoz.....	8
4.5. VS Code konfigurálása C programozáshoz.....	9
5. A man (kézikönyv) lapok használata.....	9
6. Bibliográfia.....	10

1. A GNU C fordító: gcc

1.1. A gcc fordítóprogram használata

Egy C forráskód fordításának fázisai

A fordítás alatt a fordító a következő lépéseket végzi:

1. C forrás előfeldolgozása (`#include`, `#define`, makrók stb. átírása): ezt a **preprocesszor** végzi
2. Fordítás, eredménye a bináris kód (egy `.o` kiterjesztésű fájlban) – ezt a **fordító** végzi
3. A kódok összekötése, könyvtárak beillesztése: ezt a **linker** végzi

Linux rendszereken a leggyakrabban használt C fordító a GCC (GCC, the GNU Compiler Collection). A csomag nem csak a C, hanem a C++, Fortran, ADA és más nyelvekhez biztosít fordító programokat. Linuxon C fordításakor a legegyszerűbb a `gcc` paranccsal meghívni.

Ubuntu Linuxon a C fejlesztési környezetet így telepítjük (más, újabb verziójú `gcc` vagy `make` is megfelel):

```
$ sudo apt-get update
$ sudo apt-get upgrade
$ sudo apt-get install build-essential
$ gcc -v
...
gcc version 7.4.0 (Ubuntu 7.4.0-1ubuntu1~18.04.1)
$ make -v
GNU Make 4.1
...
$
```

A következő példákban a `teszt.c` nevű fájlt fogjuk használni. Nézzük át a forráskódot!

Töltsük le a Moodle honlapról az `OPSYS.tgz` fájlt a Downloads könyvtárba, majd egy terminálban pakoljuk ki a saját könyvtár alá:

```
$ cd ~
$ tar -xzf ~/Downloads/OPSYS.tgz
```

A továbbiakban egyelőre terminálban fogunk fordítani, szövegszerkesztőként ennél a labornál bármilyen programot lehet használni, pl. a gedit-et vagy a VS Code-ot.

A későbbi laborokon ajánlott a VS Code-ot vagy más fejlesztő környezetet használni, és nem egyszerű kis szerkesztőket (lásd alább).

Az első labor anyagai a `lab1` könyvtárban találhatóak:

```
cd ~/OPSYS/lab1
```

A legegyszerűbb fordítási parancs a `gcc` meghívása a forrásprogrammal mint argumentum. Írjuk be az alábbi parancsot:

```
$ gcc teszt.c
```

, ilyenkor a kimenet az `a.out` végrehajtható állomány, amely az:

```
$ ./a.out
```

paranccsal azonnal fut (tehát a fordító alapértelmezetten beállítja az `x` jogot is). A program egy egész számot fog bekérni, majd kiír egy futási eredményt.

A köztesen előállított munka fájlokat, mint pl. a `.o` végződésű kód fájlok a `gcc` ilyenkor nem hozza létre.

A C fordítók első lépésben kimenetként a kód állományt (*object file*) állítják elő, utána kapcsolják össze a kódot a szükséges könyvtárakkal és állítják elő a végrehajtható állományt (*executable*).

```
$ gcc teszt.c -o teszt
```

parancsra a végrehajtható állomány neve `teszt` lesz, tehát így indítható:

```
$ ./teszt
```

A Linux alatt a futtatható fájlok ELF (*Executable and Linkable Format*) formátummal rendelkeznek. Az érdeklődők pl. a következő weblapon olvashatnak erről részleteket (opcionális, nem annyira fontos most tudnunk, hogyan vannak ezek kódolva): <https://linux-audit.com/elf-binaries-on-linux-understanding-and-analysis/>

A `-c` opció hatására a linker nem hívódik meg, és a `gcc` csak a fordítást végzi el. Ilyenkor a lefordított kód állomány neve `.o` kiterjesztést kap. Ez a kód nem futtatható, egyetlen C fájl (vagy C modul) fordított gépi kódját tartalmazza:

```
$ gcc -c teszt.c
$ ls *.o
```

```
teszt.o
$
```

A labor gyakorlat `hexdump` könyvtárában két C fájlban megírt kis programot találunk (állapítsuk meg, mit végez a program?).

A két C fájlban megírt programot az alábbi paranccsal fordítjuk:

```
$ gcc dump.c hex.c -o hex
```

Felsoroljuk a C fájlakat és opcionálisan megadhatjuk a futtatható fájl nevét. Ha nem adjuk meg, akkor ebben az esetben is `a.out` lesz a kimenet neve. Nyilván ilyenkor csak egyiküknek kell a `main()` függvényt tartalmaznia. Amennyiben csak fordítani akarom őket végrehajtható állomány létrehozása nélkül, a parancs:

```
$ gcc -c hex.c dump.c
```

A kimenet ebben az esetben két `.o` fájl (ellenőrizzük!). Ezek után amennyiben össze akarom kötni őket a linker futtatásával:

```
$ gcc dump.o hex.o -o hex
```

az eredmény a `hex` végrehajtható állomány.

1.1.1. C könyvtárak használata

A fordító alap beállításaként használja a standard könyvtárat, ennek a neve Linuxon `libc`. A standard könyvtár fájlok, sztringek, memória kezeléséhez, és más operációs rendszer szolgáltatásokhoz nyújt támogatást (pl. az I. éves C kurzusban használt függvények).

A saját rendszerünk `libc` verzióját így írhatjuk ki:

```
$ ldd --version
ldd (Ubuntu GLIBC 2.31-0ubuntu9.3) 2.31
Copyright (C) 2020 Free Software Foundation, Inc.
```

Az `ldd` a `libc` egyik társ programja, ki tudja írni egy futtatható állományról, hogy az milyen könyvtárakat használ:

```
$ which cat
/usr/bin/cat
$ ldd /usr/bin/cat
linux-vdso.so.1 (0x00007ffc1c756000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f5b3192a000)
/lib64/ld-linux-x86-64.so.2 (0x00007f5b31b4c000)
```

Látható, hogy a standard könyvtár helye a konkrét gépen: `/lib/x86_64-linux-gnu/libc.so.6`.

Amennyiben a függvényhívások **külső könyvtárakat** is kérnek a standard könyvtáron kívül, ezek nevét meg kell adni egy opcióval. A C könyvtárak statikusak vagy dinamikusak lehetnek: a statikusakból kivett könyvtárkódok bekerülnek a saját futtatható fájlba, a dinamikusak külön töltődnek a memóriába, és több futó kód használhatja őket.

A Linux statikus C könyvtárai `.a` kiterjesztésűek, a standard C könyvtár neve `libc.a`, a matematikai függvényeket tartalmazó könyvtár a `libm.a`, a dinamikus könyvtárak pedig `.so` végződésűek. A Linuxon a könyvtárak nagy részének helye a `/lib` vagy `/usr/lib` és- vagy `/usr/local/lib` könyvtárak. A `gcc` implicit dinamikus könyvtárakat linkel, a statikusak használatához meg kell adni a `-static` opciót.

Például, ha programjainkban használjuk a `double pow(double x, double y);` függvényt, amely a matematikai könyvtárban található, akkor fel kell sorolni a bementi állományok közt a `libm.a` könyvtárat. Szerkesszük meg a `teszt.c` fájlt, és a 79. sorból vegyük ki a megjegyzést, tegyük megjegyzést a 80. sorba. Ha most újra fordítjuk a programot hibát fog adni:

```
$ gcc teszt.c -o teszt
/tmp/ccWiXXkT.o: In function `main':
teszt.c:(.text+0x1cc): undefined reference to `pow'
collect2: error: ld returned 1 exit status
$
```

Megjegyzés: jelenleg az Ubuntu 18.04 és 20.04 rendszereknél ez a hiba nem biztos, hogy megjelenik: az alapértelmezett beállításokban gyakran benne van a matematikai könyvtár.

A `libm.a` matematikai könyvtár linkelését a `-l` opcióval adhatjuk meg:

```
$ gcc teszt.o -lm -o teszt
```

A `-lnev` opció a `libnev.a` alakú könyvtárakat is beveszi a linknél, nem csak a standard könyvtárat. A `-lm` opciónak **később** kell megjelennie a parancssoron mint az a C fájl amelyik használja.

A `gcc` parancssorán a linkelés műveleténél sok esetben számít az összekötött összetevők (`.o` fájlok és könyvtárak) sorrendje. Pl. a fenti esetben is a `-l` opciónak követnie kell azt az `.o` vagy `.c` fájlt amelyikben a hivatkozott `pow()` függvény található. Pl. egyszerű kompilálásnál:

```
$ gcc -lm teszt.c
/tmp/ccNh7yex.o: In function `main':
teszt.c:(.text+0x101): undefined reference to `pow'
collect2: error: ld returned 1 exit status
$ gcc teszt.c -lm
$
```

1.1.2. Fejléc állományok használata

A fejléc (*header*) állományokat első szinten a `gcc` a `/usr/include` könyvtárban, illetve annak alkönyvtáraiban keresi, ha:

```
#include <stdio.h>
```

alakban használjuk az `include` direktívát. Amennyiben a

```
#include "teszt.h"
```

alakot használjuk, a munka könyvtárban keres. Ha pedig azt szeretnénk, hogy más könyvtárakban is keressen, akkor ezt a parancssoron a `-I` opcióval kell megadni:

```
$ gcc -I/opt/include teszt.c -o teszt
```

1.1.3. Néhány fontos gcc opció

Az alábbi táblázat bemutat néhány fontos gcc opciót:

<code>-c</code>	csak fordítás történik, nem fut le a linker
<code>-o név</code>	a kimeneti futtatható állomány neve
<code>-Wall</code>	Minden figyelmeztető üzenetet kiír (<i>warnings</i>), ajánlott mindig használni!!!

-g	létrehozza a hibakereséshez szükséges szimbólumtáblát
-ggdb	létrehozza a hibakereséshez szükséges szimbólumtáblát a gdb, GNU debugger számára
-lkönyvtár	a linknél felhasználja a könyvtár könyvtárat is
-Ikönyvtár	ebben a könyvtárban és alatta is keresi az include állományokat
-Lkönyvtár	ebben a könyvtárban és alatta is keresi a szükséges könyvtárakat
-Ddef=ertek	definiálja a def makrót vagy konstanst <code>ertek</code> értékkel
-E	csak a preprocesszort futtatja le

A **-D** opció az = jellel megadott sztringeket két részre bontja, és úgy futtatja le, mintha egy `#define` direktívában futtatnánk:

```
#define def érték
```

Ha az = jel és a másodig rész hiányzik, akkor csak a `def` szimbólumot hozza létre.

Pl. a ha a C kódba az alábbi sorokat illesztjük (`teszt.c` 54. sor):

```
#ifdef DEBUG
    printf ("i=%d értéket olvastunk be\n", i);
#endif
```

és

```
$ gcc -DDEBUG -ggdb teszt.c -lm
```

paranccsal kompilálunk, akkor a `printf()` le fog futni, ha elhagyjuk kompiláláskor a `-D` opciót, akkor viszont nem fut le.

Feladat: Módosítsuk a `hello.c` programot úgy, hogy amennyiben `-DDEBUG` opcióval fordítjuk, akkor az alábbi kimenetet adja: *Helló-Belló!* Tehát:

```
$ gcc hello.c -o hello
$ ./hello
Helló!
$ gcc -DDEBUG hello.c -o hello
$ ./hello
Helló-Belló!
$
```

A C fordító hibákat (*error*) és figyelmeztetéseket (*warning*) írhat ki fordítás közben. Hibák esetén nem generálódik kód, csak figyelmeztetés esetén viszont igen.

A **-w** opció meghatározza, hogy milyen figyelmeztetéseket írjon ki a fordító. Ha **-Wall** formában használjuk, a fordító olyan figyelmeztető üzeneteket, amelyeket egyébként nem írna ki, mert nem veszélyeztetik a program végrehajtását. Ilyen pl. egy nem használt, de definiált változó. Ha a `test.c` 47. sorát megjegyzésre tesszük, akkor a:

```
$ gcc teszt.c
```

lefut figyelmeztetés kiírása nélkül, de a

```
$ gcc -Wall teszt.c
```

jelezni fogja a nem használt változót. Próbáljuk ki!

2. Programok fordításának automatizálása: a make parancs

A folyamatos parancs gépelés elkerülése végett, hosszú fordító parancssorozatok automatizálhatóak a `make` nevű fejlesztő eszköz segítségével.

A `make` használatát a `hex.c` és `dump.c` példaprogramokon keresztül mutatjuk be laboron. Később külön anyagban átvesszük a `make` részletes használatát. Egyelőre kis, egy fájlt tartalmazó programokkal dolgozunk és így nem lesz erre szükség.

A `make` használatának részletes leírását magyarul megtaláljuk részletesen [az \[1\] bibliográfiában megadott könyv](#) 541. oldalától kezdve.

3. A C program parancssori paramétereinek használata

A `main()` függvénynek két paramétere van, egy egész szám (`argc`: hány parancssori argumentum van) és egy mutatókból álló tömb (`argv`), a mutatók sztringekre mutatnak.

```
int main( int argc, char * argv[] ){
//...
}
```

Az `echoarg.c` program fejlécében megtalálható az `argv` szerkezete, valamint egy példa arra, hogyan írhatjuk ki a program parancssori argumentumait. Itt az `argv`-t egyszerűen tömbként használjuk (lehetne mutatóval is lépkedni rajta):

```
$ gcc echoarg.c
$ ./a.out alma szilva cseresznye
./a.out
alma
szilva
cseresznye
$
```

Az első argumentum, amelyik mindig létezik a program neve, így az `argc` mindig legalább 1.

Feladat: írjunk egy programot, amelyik leteszteli, hogy két argumentuma van (ha nincs így, hibaüzenettel kilép), és kiírja, hogy ezek egyenlőek-e mint sztringek. A `strcmp()` függvény használatához a `#include <string.h>` fejléct kell használni. A függvény prototípusa megtekinthető a:

```
$ man 3 strcmp
```

parancsal.

A C program megkapja a környezeti változókat is egy `environ` nevű globális mutatón keresztül. Ha kivesszük az `echoarg.c` program 31-33. sorok elől a magyarázatokat, megtekinthetjük ezek kiírását.

4. Unix szövegszerkesztők C programozáshoz

A C programokat szerkeszthetjük *egyszerű szövegszerkesztőkkel*, vagy *fejlesztési környezetekkel*.

C szövegek szerkesztéséhez **parancssorból** kis programok esetén megteszi a **nano** vagy **joe** is.

Grafikus felület alatt egyszerű feladatokhoz a **Kate**, **Gedit** vagy egyéb programfejlesztésre alkalmas szövegszerkesztőket lehet használni. Használhatjuk az **mc** beépített szerkesztőjét (**mcedit**).

A C programok haladó szintű szerkesztéséhez akár terminálban, akár grafikus környezetben hagyományosan a *Vim* vagy *Emacs* szövegszerkesztőket használhatjuk, ezekről részletesen tájékozódni a Moodle honlapon lehet.

Jelenleg jól használhatóak erre a feladatra a modern JavaScript alapú fejlesztők is, mint az *Atom* vagy *Visual Studio Code*.

4.1. A vi és vim használata C programok szerkesztésére

A *vi* a Unix alapú rendszerek alap szövegszerkesztője, kifejezetten arra van fejlesztve, hogy terminálban, csak gépeléssel gyorsan lehessen bonyolult szövegszerkesztési feladatokat megoldani, automatizálni. Minden Linux disztribúción a legkisebb installáláskor is megtalálható. **Bevezető anyagok a Moodle honlapon találhatóak.** Tanulása nem könnyű, sok időt vesz igénybe, viszont érdekes és a befektetett munka kifizetődik.

Az alábbiakban néhány fontos vim parancsot tekintünk át C programok szerkesztéséhez, **ha túl vagyunk a bevezetőn(!)**. Vim bevezető anyagok találhatóak a Moodle honlapon.

A forráskódban a függvényt kezdő kapcsos zárójelt mindig a következő sor első karakterére írjuk, a záró kapcsost pedig szintén az első karakterre.

Parancsmód (command)	
{ }	Normál szerkesztésnél paragrafusról paragrafusra ugratja a kurzort: C szerkesztésnél üres sortól üres sorig. Ezért ajánlatos a forrásban a logikailag összefüggő részek közt egy-egy üres sort hagyni.
[[]]	Függvényről függvényre ugrál a kurzor (első karakteren levő kapcsosra).
%	A zárójelek, illetve kapcsos zárójelek helyes zárását lehet ezzel ellenőrizni (a kurzort az első zárójelre kell helyezni, és lenyomni a %-at, ekkor a neki megfelelő záróra ugrik). Ezzel tk. a kapcsosok közt ugrálni is lehet.
m betű	Markerek beállítása: ha ide-ida kell ugrálni az állományban, akkor az ma paranccsal beállíthatunk egy markert, utána pedig a 'a paranccsal visszatérhetünk hozzá. 26 marker állítható be a kisbetűknek megfelelően.
gd	Amennyiben egy név fölött van a kurzor (pl. egy struktúra neve) akkor a struktúra deklarációjához ugrik. Vissza úgy lehet ugrani ha kétszer leütjük a '' parancsot. Ez minden ugrálásra igaz. Előzőleg futtatni kell a ctags parancsot, amely előállítja az ugráshoz szükséges címkéket.

A ctags címkék

A **ctags** Linux paranccsal állíthatóak elő a forrásoknak megfelelő címkék

```
$ ctags *.c
$ #rekurzív használat :
$ ctags -R *.c
```

Egyéb ötleteket a C állományok szerkesztéséhez a [C editing with vim howto](#) nevű dokumentumból lehet elolvasni.

4.2. Az Emacs használata C programok fejlesztéséhez

Az *Emacs*-ot kifejezetten programozási, és elsősorban C-ben való programozási feladatokhoz fejlesztették. Első fejlesztője [Richard Stallmann](#), a nyílt forráskódú mozgalom atyja.

A Moodle honlapon található [egy szakkikk](#), ami érdekes bevezetőt nyújt, valamint néhány link bevezető segédanyagokhoz, olvassuk át ezeket! További segédletek találhatóak az *Emacs* honlapján és több helyen az Interneten.

4.3. Fejlesztői környezetek használata C programozáshoz

Több fejlesztői környezet használatát lehet igénybe venni Linux alatt. Ezek általában azonnal telepíthetők az általunk is használt Ubuntu disztribúció alá.

A fontosabbak:

- egyszerűbbek: Gnome alatt az *Anjuta* (apt csomag neve; `anjuta`) és a *Code::Blocks* (csomag név: `codeblocks`),
- KDE alatt *KDevelop* fejlesztő környezetet (`kdevelop`),
- az *Eclipse* és *Netbeans* is alkalmas C fejlesztésre (`eclipse` és `netbeans`). Mindkettő telepíthető Ubuntu csomagként, a legújabb változatok is általában megtalálhatóak külön ppa csomagban. Mivel a laboron nagyon egyszerű kis programokat írunk, nem ajánlott ezeknek a behemót környezeteknek a használata.
- Modern szerkesztők, mint az *Atom* vagy a *Visual Studio Code* is jól használhatóak C programok szerkesztéséhez.

Ubuntu alá bármelyiket be lehet installálni a grafikus környezet *Software Center* vagy *Synaptic package manager* alkalmazásaival, illetve terminálból az `apt-get` paranccsal. Installálás előtt frissítsük a rendszert a grafikus felületről *Software Updater*-el vagy terminálból az `apt-get` paranccsal:

```
$ sudo apt-get update
$ sudo apt-get upgrade
```

A kiválasztott programot fel lehet installálni a grafikus felületről, vagy terminálból, a megfelelő parancsokat meg lehet keresni az Ubuntu honlapján, pl. a *Code::Blocks* installálásához:

```
$ sudo apt-get install codeblocks
```

4.4. A *Code::Blocks* beállítása C programokhoz

Bár a *Code::Blocks* környezetet már nem fejlesztik, egyes tantárgyainknál még használják, ezért röviden vázoljuk a laboron szükséges C fejlesztéshez a beállításokat.

A *Code::Blocks*-ban projektet kell létrehozni, másképpen bizonyos dolgok nem fognak működni, pl. a hibakeresés. A labor anyagához a következőt végezzük az új projekt létrehozásánál lefutó kérdés sorozatnál:

1. a projekt típusa „Console Application”,
2. a nyelv C (és nem C++ !!!),
3. kiválasztjuk a projekt nevét (a projektet az szerkesztő egy `.cbp` állományba teszi, ennek automatikusan ad nevet),
4. kiválasztjuk a könyvtárat ahova a projekt kerül (a saját könyvtárunk alatt),
5. kiválasztjuk a fordítót (GNU GCC Compiler),
6. kiválasztjuk „Create DEBUG configuration” : mert akarunk majd hibát keresni, ez a normál munkamenet,

7. a „create Release”-t is bejelölve hagyhatjuk, ide akkor fordítunk, ha véglegesen elkészült az alkalmazás.

Látható a létrehozás folyamán, hogy a kimeneti fájlok `bin` (futtatható) és `obj` (kód fájlok) könyvtárakba kerülnek, és ezek alatt a `Debug` vagy `Release` könyvtárakba kompilálási módtól függően.

Létrehozáskor automatikusan létrejön egy `main.c` nevű állomány, ami a főprogramot tartalmazza majd (itt van a `main()` függvény **amiből csak egy lehet a projektben**).

További beállításokat a teljes rendszerre a *Settings* menüből, a projektre a *Projekt/Properties* menüpontban lehet elvégezni.

A *Build* menüpont tartalmazza a kompilálási parancsokat, a *Debug* pedig a hibakeresés esetén használandókat – ezek a parancsok kint vannak grafikusán is egy menüsávban, akár más fejlesztő környezeteknél, hasonló elrendezésben.

Ubuntu vagy más Gnome alapú rendszer alatt a *Settings/General settings* menüpontban állítsuk a *Terminal to launch console programs* opciót `gnome-terminal` típusra, ez a megszokott terminálunk. A parancs módú programjaink egy ilyen terminál alatt fognak elindulni mint kontrolláló terminál.

A rendszer alatt lehet használni hibakeresésre a `gdb`-t. Abban a forrás sorban, ahol meg akarjuk állítani a programot töréspontot teszünk (jobb kattintás és *Add breakpoint*), utána *Debug* módban indítjuk a programot (Fent-Bal oldali menüsáv, *Run to cursor* – itt találhatóak a hibakeresésnél megszokott menüpontok). Megállás után a szerkesztő alsó ablakában (*Log&others* adhatjuk ki a már ismert GDB parancsokat a változók listázásához).

További információk *Code::Blocks*-al való hibakereséshez [ezen a linken](http://wiki.codeblocks.org/index.php/Debugging_with_Code::Blocks) (http://wiki.codeblocks.org/index.php/Debugging_with_Code::Blocks).

4.5. VS Code konfigurálása C programozáshoz

A népszerű *Visual Studio Code* beállítása C/C++ programozáshoz megtalálható a következő linken: <https://code.visualstudio.com/docs/languages/cpp>.

Az alábbi kiegészítőt telepítsük a VS Code-ban, az *Extensions* menüpontot választva:

- a. C/C++ for Visual Studio Code, Microsoft**, C/C++ IntelliSense, debugging, and code browsing.

Jelenleg a laboron a VS Code-ot ajánlott használni, sok előnye mellett nagyon jól kezeli a C debugger futtatását grafikus környezetből.

5. A man (kézikönyv) lapok használata

A laboron a programozás C-ben történik. Linux vagy UNIX alatt a különböző C könyvtárakban található függvényekről a már ismert `man` lapok segítségével kapunk információt. A C függvények a `man` lapok második és harmadik szekciójában vannak, és pedig:

1. az első szekcióban a Unix parancsok vannak,
2. a második szekcióban a rendszerhívások,
3. a harmadikban pedig a Unix C könyvtár függvényei.

Tehát:

```
$ man 1 printf # a printf shell parancs leírás
```

```
$ man 3 printf # a printf() C könyvtári függvény leírása
```

parancsokkal lehet pontosan hivatkozni a megfelelő szekcióra, ha a név több szekcióban van meg. Ha a név csak egy szekcióban van meg, akkor az 1,2,3 számok nem szükségesek, pl:

```
$ man fork # a fork() rendszerfüggvény leírása
```

Ha nem tudjuk, melyik fejezetben keressünk egy függvény leírást, akkor az `apropos` nevű shell parancs segíthet. Pl. ha tudni akarjuk, hogy milyen `man` lapokon találunk szemaforokkal kapcsolatos függvényeket (a szemafor egy rendszerprogramozásban használt szerkezet), akkor:

```
$ apropos semaphore
ipcrm (1)          - remove a message queue, semaphore set or shared
memory id
sem_close (3)      - close a named semaphore
sem_destroy (3)    - destroy an unnamed semaphore
sem_getvalue (3)   - get the value of a semaphore
sem_init (3)       - initialize an unnamed semaphore
sem_open (3)       - initialize and open a named semaphore
sem_overview (7)   - overview of POSIX semaphores
sem_post (3)       - unlock a semaphore
sem_timedwait (3)  - lock a semaphore
sem_trywait (3)    - lock a semaphore
sem_unlink (3)     - remove a named semaphore
sem_wait (3)       - lock a semaphore
semctl (2)         - System V semaphore control operations
semget (2)         - get a System V semaphore set identifier
semop (2)          - System V semaphore operations
semtimedop (2)     - System V semaphore operations
~ $
```

Az `apropos` kulcsszó szerint (pl.: *semaphore* a fenti példában) keres a `man` lapok első, leíró sorában. Itt megadhatunk reguláris kifejezést (`-r` opció) vagy shell mintát is (`-w` opció), lásd `man apropos`.

6. Bibliográfia

1. Bányász Gábor, Levendovszky Tihamér: Linux programozás, 2. kiadás, Szak Kiadó, 2012, az A és B függelék tartalmazza a fejlesztő eszközöket. A könyv kivehető az [egyetem könyvtárából](#).

A könyvnek van egy régebbi kiadása: Bányász Gábor, Levendovszky Tihamér: Linux programozás, Szak Kiadó, 2003, [ebből is több példány van a könyvtárban](#).

2. Brian Gough: An Introduction to GCC, Network Theory Limited 2004. Letölthető a Moodle honlapról.

3. Pere László: UNIX-GNU/Linux programozás C nyelven, Kiskapu Kiadó, 2003.